API Design & Best Practices

> Part of the Python API Backend curriculum. This document covers data loading and caching, error-handling patterns (including failing closed when serving pipeline output), API design conventions (response envelopes, reporting the scope actually served), and security fundamentals for a Python web API.

1. Data loading and caching

Since the API's job is to read already-computed results (not recompute them), a common pattern is a small loader function per data source, with in-memory caching so repeated requests don't re-read from disk every time:

```
from functools import lru_cache
import pandas as pd

@lru_cache(maxsize=1)
def load_dataset() -> pd.DataFrame:
    return pd.read_csv("path/to/computed_output.csv")
```

`@lru_cache(maxsize=1)` memoizes the function's return value — the first call actually reads the file; every call after that (for the process's lifetime) returns the same cached object instantly. This is the same idea as any application-level cache, just at Python-function granularity. `maxsize=1` means "remember only the single most recent call" — appropriate here since the function takes no arguments, so there's only ever one possible cached value.

1.1 Beyond `lru_cache`: caching across multiple processes

`lru_cache` is **per-process, in-memory** — if your API runs as multiple worker processes (common in production, see Deployment, §2), each worker has its own independent cache, and none of them share it. This is fine for read-only data loaded from a file that doesn't change while the process is running. If you need a cache that's shared across multiple processes/machines, or that needs to expire on a schedule (a time-to-live), an external cache like **Redis** is the standard tool — see the persistence options table for where an in-memory cache fits relative to other persistence choices.

1.2 Cache invalidation

The classic hard problem: if the underlying data file changes while the API process is already running, an `lru_cache`'d loader won't notice — it keeps returning the stale, first result forever. For data that genuinely changes while a service is running, you need an explicit invalidation strategy: a time-based expiry, a file-modification-time check before deciding whether to reuse the cache, or simply restarting the service after new data is published (often the simplest, most reliable option for a periodically-refreshed batch pipeline like the one this series describes).

2. Error handling

2.1 A custom exception for "the data isn't ready yet"

Since an API server's data dependencies might not exist yet (an earlier processing stage hasn't produced them), define a specific exception for that case and map it to the `503` status code discussed in Fundamentals, §2.3 — distinct from a generic crash, and distinct from a `404` (the *endpoint* is valid; the *data* just isn't there yet):

```
from fastapi import HTTPException
import os

class DataNotFoundError(Exception):
    def __init__(self, message: str):
        self.message = message
        super().__init__(message)

@lru_cache(maxsize=1)
def load_dataset() -> pd.DataFrame:
    if not os.path.exists(PATH):
        raise DataNotFoundError(f"{PATH} not found — has the upstream pipeline run yet?")
    return pd.read_csv(PATH)

@app.get("/items")
def list_items():
    try:
        df = load_dataset()
    except DataNotFoundError as e:
        raise HTTPException(status_code=503, detail=e.message)
    ...
```

2.2 Required vs. optional dependencies

Not every endpoint needs every data source to be present — an endpoint whose result degrades gracefully with partial data (e.g., a chart that can render with one series absent) can treat a missing *optional* dependency as `None` rather than failing the whole request:

```python
@app.get("/combined-view")
def get_combined_view():
    try:
        primary = load_primary_dataset()   # required — a 503 if missing is correct
    except DataNotFoundError as e:
        raise HTTPException(status_code=503, detail=e.message)

    try:
        secondary = load_optional_dataset()   # optional — degrade gracefully
    except DataNotFoundError:
        secondary = None

    return build_response(primary, secondary)
```

Only truly required dependencies should turn into a hard `503` for the whole endpoint — distinguishing "required" from "optional" per-endpoint (not per-dataset globally) reflects the reality that the same underlying dataset might be essential for one endpoint and merely a nice-to-have enhancement for another.

2.3 Fail closed: a 503 that names the cause, never a 200 with nulls

When an API serves the output of a data pipeline, the dangerous failure is not the crash — it is the response that *looks* fine. A missing statistic returned as `null`, a stale file served as current, a `NaN` rendered as a zero on a chart: each is a 200 that a client will happily plot. Make the loader layer **fail closed**: any file that cannot be trusted raises the "data isn't ready" exception from §2.1, and the router turns it into a `503` whose `detail` names the specific cause. The conditions worth checking explicitly:

Condition	Why it must be a 503, not a 200
The file does not exist	The pipeline stage has not run
The file holds the stage's own "unavailable" marker (e.g. an `unavailable_reason` field)	The pipeline *knew* the result was not publishable and said so; do not hide that
`schema_version` is not one this API understands	A newer pipeline wrote a shape this code would misread
Any number is non-finite (`NaN`, `Infinity`)	Not valid JSON, and a chart would draw it as nothing or as zero
A CSV disagrees with the metadata JSON describing it	Two halves of one result from different pipeline runs

```
class ClimateDataUnavailable(DataNotFoundError): ...


@lru_cache(maxsize=None)
def _read_json(name: str, directory: str) -> dict:
    path = os.path.join(directory, name)
    if not os.path.exists(path):
        raise ClimateDataUnavailable(f"{name} has not been generated yet (the pipeline has not
        run)")
    doc = json.load(open(path))
    if doc.get("schema_version", SUPPORTED) != SUPPORTED:
        raise ClimateDataUnavailable(f"{name} has schema_version {doc['schema_version']!r}")
    if doc.get("unavailable_reason"):
        raise ClimateDataUnavailable(f"{name} is unavailable: {doc['unavailable_reason']}")
    _require_finite(doc, name)          # walks dicts/lists; raises on NaN/inf
    return doc
```

The API reads, the pipeline computes. A derived-analysis domain (fitted statistics, harmonised indicators, scenario translations) should be computed once, by the pipeline, and served as stored — never recomputed per request, and never fitted on the fly "just for this query." An endpoint that accepts a parameter which would require a new fit (an arbitrary start year) should instead accept only the values the pipeline pre-computed, and answer 422 for the rest. The API must also not *import* the pipeline's code: the two meet at the files, so each can be versioned and deployed independently (Architecture Overview, §2.2). A published fit should be attached to a response only when its window matches the data the response actually carries — otherwise a slope computed on 1850–2024 gets displayed next to a 1990–2024 chart.

3. API design conventions

3.1 Resource naming

Use plural nouns for collections and singular identifiers for specific resources, and keep the hierarchy in the URL path meaningful:

```
GET /items                    # a collection
GET /items/{item_id}          # one specific item
GET /items/{item_id}/history  # a sub-resource of a specific item
```

Avoid verbs in the URL path (`/getItem`, `/fetchItems`) — the HTTP method already conveys the action; the path should describe *what*, not *how*.

3.2 Versioning

An API's contract will eventually need to change in a way that breaks existing consumers. Common strategies:

- **URL path versioning** (`/v1/items`, `/v2/items`) — simple, highly visible, easy for a consumer to pin to a specific version explicitly.
- **Header-based versioning** (a custom `Accept` or `API-Version` header) — keeps URLs stable but is less discoverable and harder to test manually in a browser.

For a small, internally-consumed analytics API where the frontend is deployed in lockstep with the backend, formal versioning is often unnecessary early on — but it's worth deciding *deliberately* that you're deferring it, rather than never considering it and being surprised later when a breaking change has no clean rollout path.

3.3 Pagination

Any endpoint that could return an unbounded number of rows should support pagination, rather than returning everything in one response:

```
@app.get("/items")
def list_items(page: int = 1, page_size: int = 20):
    start = (page - 1) * page_size
    end = start + page_size
    return {
        "items": all_items[start:end],
        "page": page,
        "page_size": page_size,
        "total": len(all_items),
    }
```

For a bounded, small analytics dataset (a fixed, known number of entities/rows), pagination is often unnecessary — but the moment a collection's size is driven by external data growth (more entities added over time, more historical periods accumulating) rather than a fixed project scope, revisit whether it should be paginated before it becomes a real problem for response size and client render time.

3.4 Rate limiting

For a public-facing API, or one with meaningfully expensive endpoints, **rate limiting** (capping how many requests a given client can make in a time window) protects the service from being overwhelmed — accidentally, by a buggy client in a retry loop, or deliberately. Libraries like `slowapi` add this to a FastAPI app declaratively; for an internal API with a small, known set of trusted consumers, this is often lower priority than the other topics in this document, but worth being aware of as the API's audience grows.

3.5 A response envelope for context that must travel with the numbers

Some data cannot be responsibly shown without its context: a licence that restricts commercial use, a citation the publisher requires, a note that the series is a preliminary release, a caveat that a relationship is interpretive. If those live only in the UI's copy, every new consumer (a second frontend, an agent, a CSV export) has to rediscover and re-type them — and eventually one of them won't. Give every response in a domain **one envelope** carrying them:

```python
class Envelope(BaseModel):
    schema_version: int
    generated_at: datetime
    note: str                    # what this is and what it is not
    caveats: list[str]           # plain-language limits
    attribution: list[Attribution]   # licence + required citation, per source
    source_vintage: dict         # which release of each source produced this
```

The values come from the stored metadata the pipeline wrote (§2.3), not from strings retyped in the router. Consumers then pass the envelope through unchanged — see MCP Tool Design, §6.2. Treat adding a field to the envelope as an additive, backward-compatible contract change (Architecture Overview, §2.2).

3.6 Report the scope you actually served

When an endpoint takes a *scope* parameter (a featured subset, an expanded set, everything) and the data behind one choice can be incomplete, the response must say what it **actually** returned, not echo what was asked for. Add a field such as `effective_scope` computed from the rows that came back:

```python
requested = "expanded"
effective = "expanded" if requested == "expanded" and set(rows_countries) > set(FEATURED) else
        "featured"
return Summary(rows=rows, effective_scope=effective)
```

Note the strictness: "expanded" is only claimed when the returned set is a *strict superset* of the smaller set. A consumer that builds a ranking ("top 10 of the expanded set") needs that field to know whether its ranking is over the population it will describe — without it, the only safe behaviour is to not render the ranking, or worse, to render it silently over the wrong set. The same applies to any "N of M" claim: return the denominator you used.

4. Security fundamentals

Even a read-only, internal analytics API benefits from a baseline security posture:

- **Input validation at the boundary** — Pydantic's type/`Literal`/`Field` validation (covered in Fundamentals, §4.2–4.3) is your first line of defense: reject malformed input before it ever reaches your business logic, rather than trying to handle every possible malformed value deep inside a function.
- **Authentication and authorization** (if the API needs them) — **authentication** answers "who is making this request" (commonly via an API key, or an OAuth2/JWT bearer token scheme); **authorization** answers "is this identity allowed to do this specific thing." A purely read-only, internal, non-sensitive analytics API (the kind this curriculum focuses on) often needs neither, but any API returning genuinely private or user-specific data needs both, deliberately designed rather than added as an afterthought. A practical consequence: if the API stays **public and unauthenticated**, do not expose *new* surface (interactive docs, schema endpoints, admin routes) through the public route until it has auth — and put *operational* capabilities (an admin page, a settings endpoint) on whichever service already owns the setting, gated at the edge by an identity-provider login policy (see Agent Deployment, §8) rather than inside this API.

- **CORS as an explicit allow-list.** Even when same-origin deployment means CORS is never triggered, list the production origin in code so a future subdomain or staging origin has to be added deliberately instead of silently working (or silently failing).
- **HTTPS/TLS** — encrypts traffic between client and server; in a real deployment this is almost always terminated at a reverse proxy or load balancer in front of the application (see Deployment, §4), not handled by the Python application itself.
- **Secrets management** — database credentials, API keys, and any other sensitive configuration values belong in environment variables (or a dedicated secrets manager in a cloud deployment), **never** hardcoded into source code or committed to version control — this is the same "configuration over hardcoding" principle from the architecture overview, §2.7. A convenient way to manage typed configuration from environment variables in a FastAPI app is `pydantic-settings`:

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    api_key: str

settings = Settings()   # reads DATABASE_URL and API_KEY from the environment automatically
```

5. Core libraries summary

Library	Used for
pandas (or whatever the data layer uses)	reading and shaping already-computed results
pydantic-settings	typed configuration read from environment variables
slowapi (or similar)	rate limiting, if/when needed
Redis (or similar)	shared, multi-process caching, if/when a single process's `lru_cache` isn't enough

See also

- Python API Backend index
- HTTP, REST & FastAPI Fundamentals — the underlying routing/validation mechanisms these patterns are built on
- Testing — how to verify all of the above actually behaves as intended